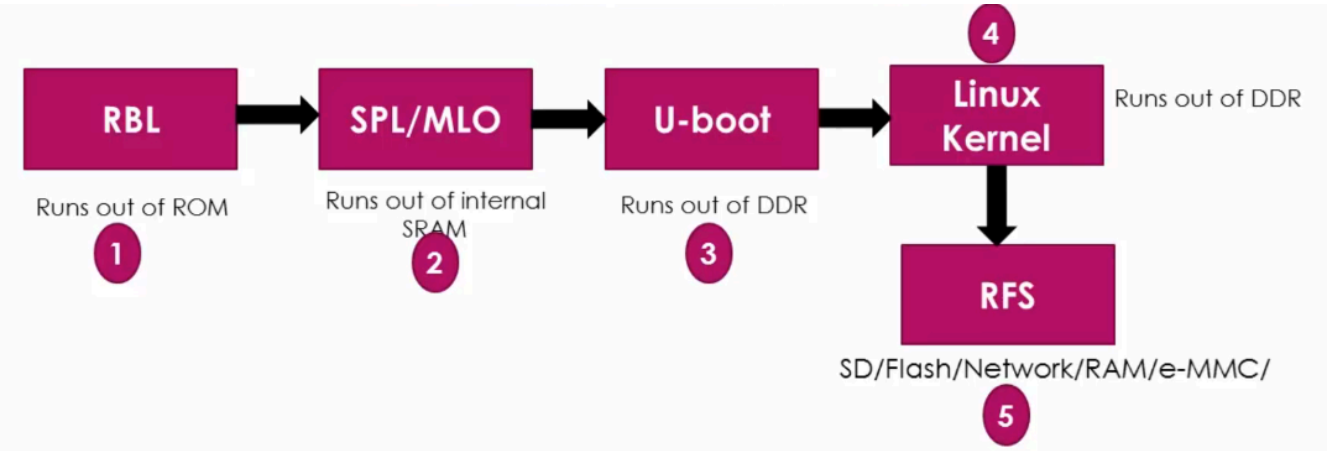
Beaglebone Linux Boot Requirements



Beaglebone'a güç verildiğinde Linux tek adımda açılmaz. Açılış, birbirini yükleyen bir **zincir** şeklinde ilerler. Her aşama bir sonrakini belleğe yükler ve kontrolü ona devreder:

Şemadaki **"Runs out of ..."** ifadeleri her aşamanın **hangi bellekte çalıştığını** gösterir. Zincirin neden bu kadar uzun olduğu da buradan anlaşılır.

> Not
>
> **Neden tek bir bootloader yok?** Açılışın ilk anında 512 MB'lık **DDR RAM henüz kullanılamaz**. DDR'ın çalışması için önce saat (clock), zamanlama ve voltaj ayarlarının yapılması gerekir ve bu ayarlar karttan karta değişir. İşlemcinin hemen kullanabildiği tek RAM, SoC'nin içindeki **çok küçük dahili SRAM**'dir. Bu yüzden önce SRAM'e sığacak kadar küçük bir program (SPL) çalışır, DDR'ı hazırlar, ardından büyük programlar (U-Boot, Kernel) DDR'a yüklenebilir.

---

1. RBL (ROM Boot Loader)

- Karta enerji verildiğinde SoC (System on Chip) üzerinde çalışan ***ilk kod parçasıdır***.
- Üretici firma (Texas Instruments) tarafından yazılır ve çip üretilirken SoC'nin içindeki **ROM** belleğe kaydedilir. ROM'da olduğu için **değiştirilemez**.
- **Nerede çalışır:** Doğrudan **ROM** üzerinden (Runs out of ROM).

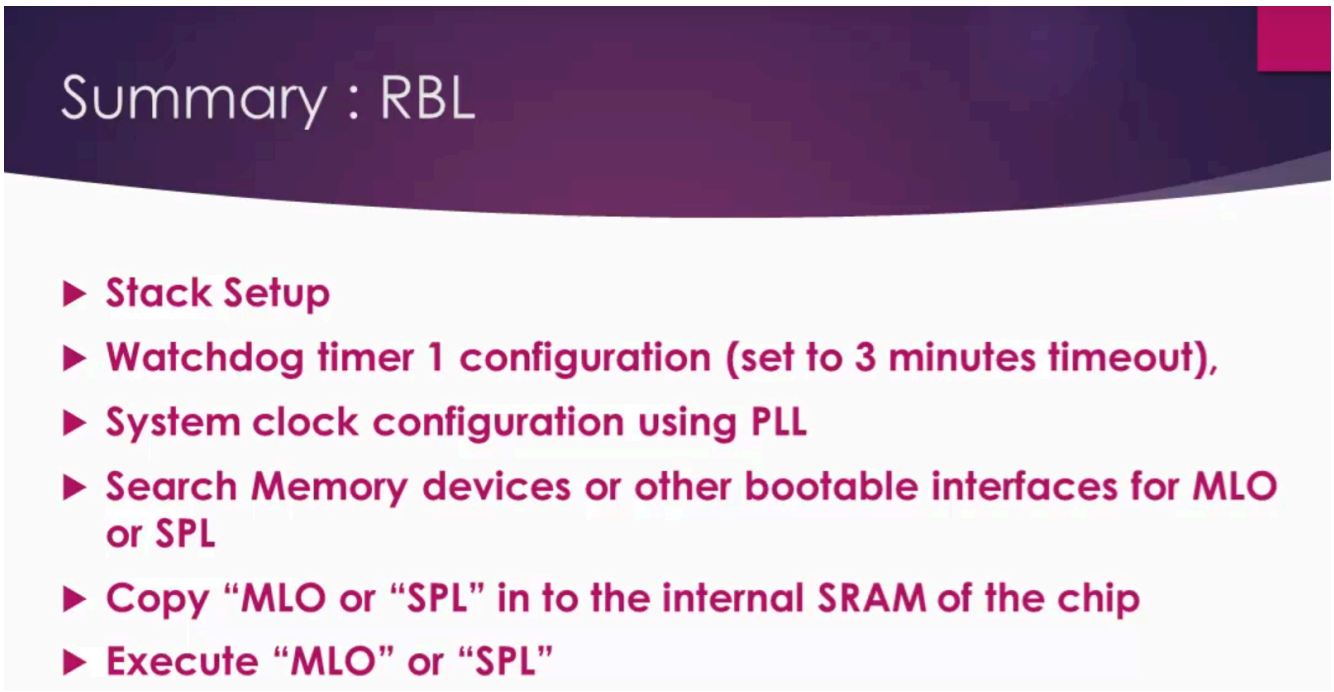
Görevleri:

1. İşlemcinin temel donanımını (saat, watchdog vb.) minimum seviyede başlatır.
2. **SYSBOOT** pinlerinin seviyesini okuyarak **boot sırasını** belirler. Beaglebone'da bu sırayı **S2 (Boot)** butonu değiştirir: - S2 basılı değilse: **eMMC** → microSD → UART → USB - S2 basılıysa: SPI → **microSD** → USB → UART
3. Bu sırayla cihazları tek tek dener ve ilk bulduğu geçerli **ikinci aşama bootloader**'ı (**MLO**) dahili SRAM'e kopyalar.

4. Kontrolü MLO'ya devreder.

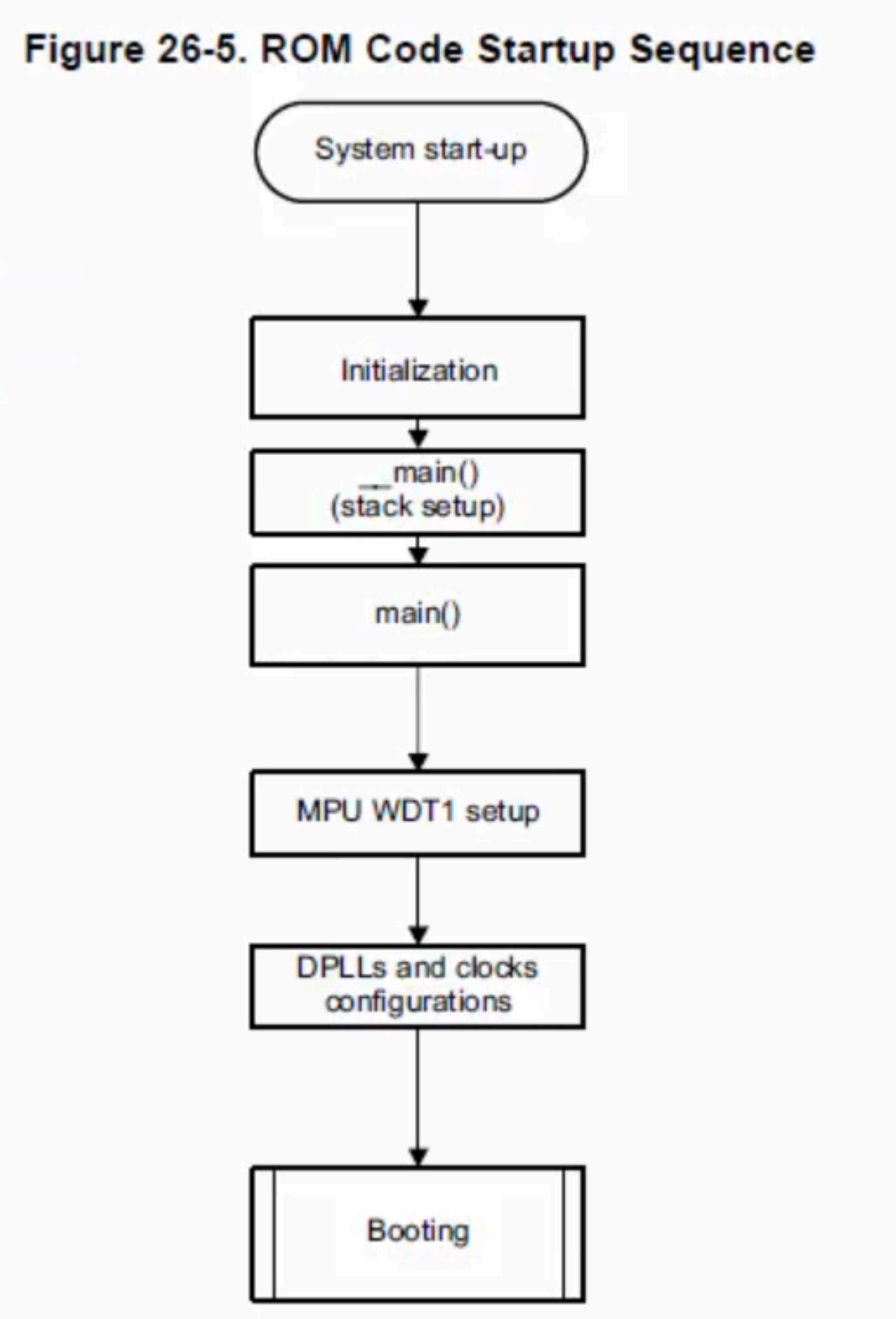
> **Not**
> SD karttan boot ederken RBL, kartın **FAT formatlı ilk bölümünde** (boot partition) `MLO` adlı dosyayı arar. Bu yüzden kendi boot SD kartımızı hazırlarken MLO dosyasını bu bölüme koyacağız.

RBL'in yaptığı işler



- **Stack Setup:** RBL'in C ile yazılmış fonksiyonları çalıştırabilmesi için önce bir **stack** (yığın) alanı gerekir. Fonksiyon çağrıları ve yerel değişkenler burada tutulur. RBL stack'i dahili SRAM'de kurar.
- **Watchdog Timer 1 (3 dakika):** Watchdog, belirli bir süre içinde "beslenmezse" (sıfırlanmazsa) sistemi **resetleyen** bir sayaçtır. Amacı, yazılım bir yerde takılırsa kartı kendiliğinden yeniden başlatmaktır. RBL bu sayacı **3 dakikaya** kurar ve açık bırakır. Bu yüzden sonraki aşamalar (SPL, U-Boot, Linux) bu watchdog'u ya kapatmalı ya da düzenli olarak beslemelidir. Yoksa kart 3 dakika sonra resetlenir.
- **System clock configuration using PLL:** İşlemcinin ve çevre birimlerinin saatlerini PLL ile kurar (aşağıda açıklanıyor).
- **MLO/SPL'i arar, SRAM'e kopyalar ve çalıştırır:** Yukarıdaki görev listesinin 2-4. maddeleri.

ROM Code Startup Sequence

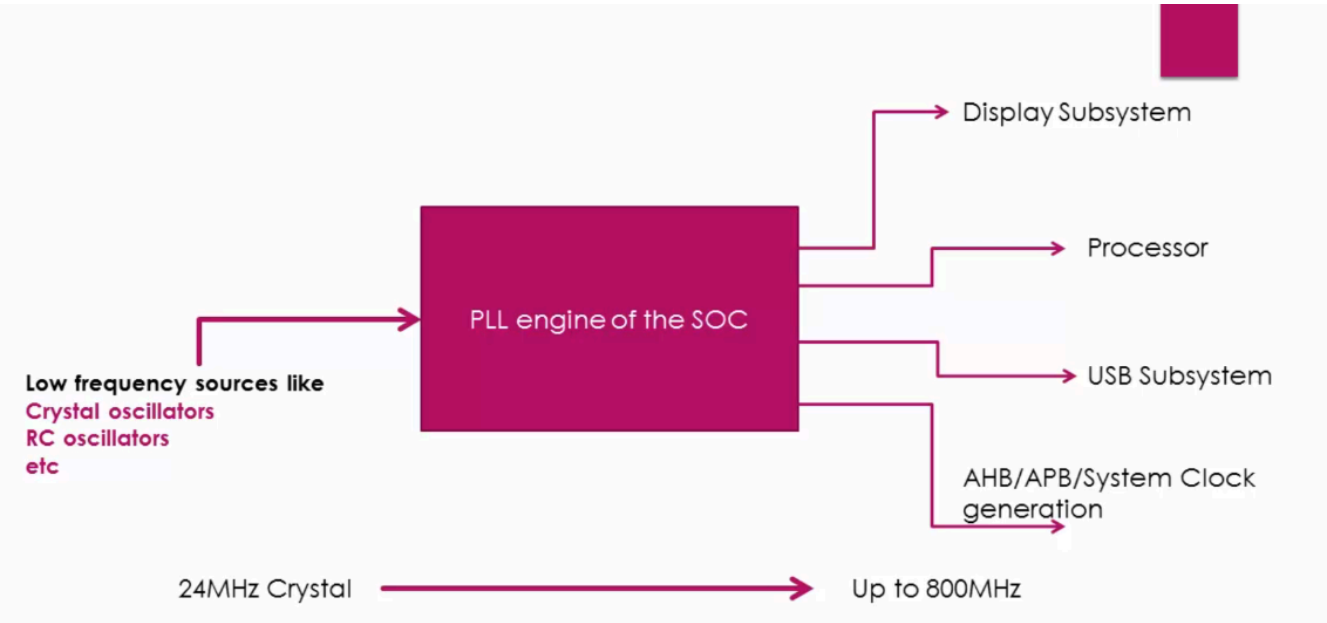


Bu akış şeması AM335x TRM'den (Figure 26-5) alınmıştır ve RBL'in açılışta yaptığı işleri sırasıyla gösterir:

1. **System start-up:** Karta güç verilir veya reset gelir.
2. **Initialization:** İşlemcinin en temel ayarları yapılır.
3. **__main() (stack setup):** Stack kurulur. Bundan sonra C kodu çalışabilir.
4. **main():** RBL'in ana fonksiyonu başlar.
5. **MPU WDT1 setup:** Watchdog Timer 1 kurulur (3 dakika).
6. **DPLLs and clocks configuration:** PLL'ler ve saatler ayarlanır.

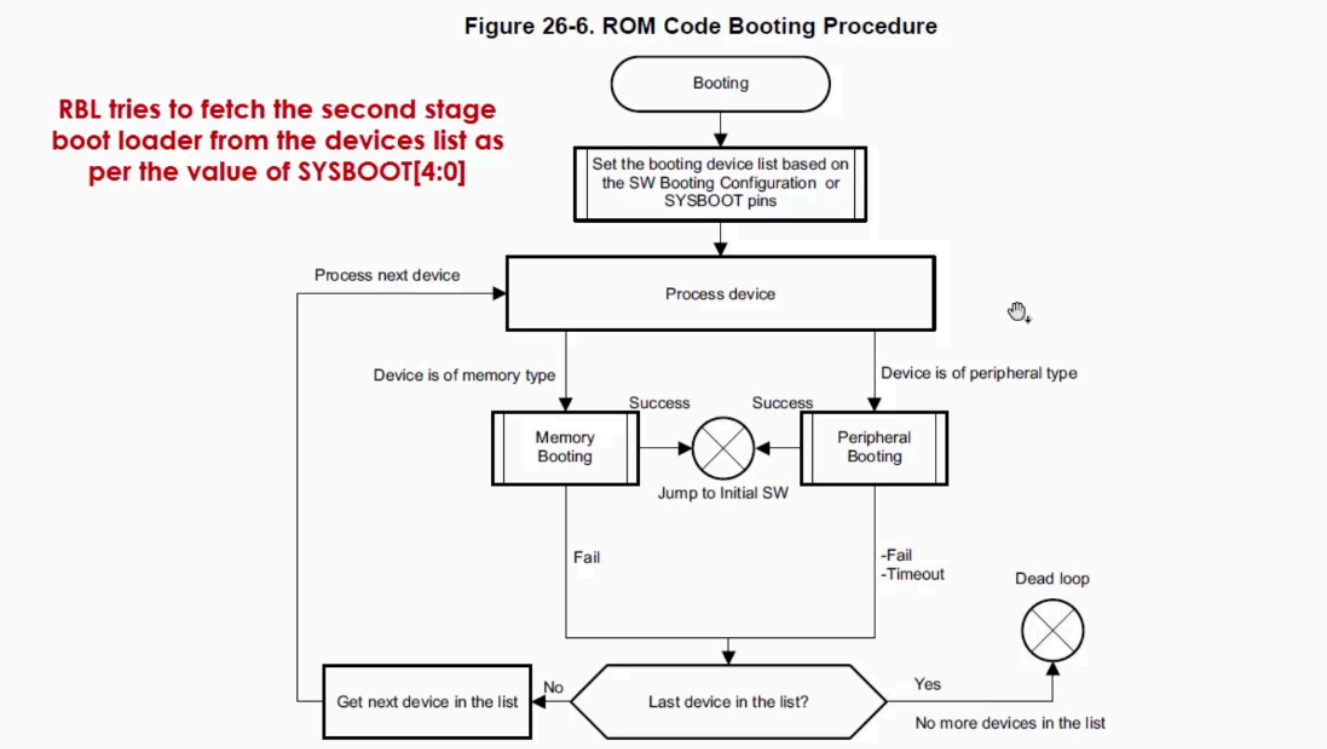
7. **Booting:** Boot cihazları aranır ve MLO yüklenir. Bu adımın detayı aşağıdaki **ROM Code Booting Procedure** şemasındadır.

PLL (Phase Locked Loop) nedir?



- Kart üzerindeki kristal osilatör **düşük frekanslıdır** (BBB'de **24 MHz**). Ama işlemci yüzlerce MHz ile çalışmak ister.
- **PLL**, SoC içindeki bir devredir. Kristalden gelen düşük frekansı **çarparak** (ve gerektiğinde bölerek) yüksek frekanslar üretir. Şemadaki örnekte 24 MHz'lik kristalden **800 MHz**'e kadar frekans elde edilir.
- Her birim farklı bir frekansa ihtiyaç duyar. Bu yüzden SoC'de birden fazla PLL vardır ve her biri farklı bir birimi besler: **işlemci**, **USB**, **ekran (display)**, **bus saatleri** (AHB/APB) gibi.
- Doğrudan yüksek frekanslı bir kristal kullanılmamasının sebebi: Düşük frekanslı kristaller ucuz ve kararlıdır. Tek bir kristalden PLL'ler ile ihtiyaç duyulan tüm frekansları üretmek çok daha pratiktir.

ROM Code Booting Procedure



Bu şema (TRM Figure 26-6), RBL'in ikinci aşama bootloader'ı (MLO) nasıl aradığını gösterir:

1. **Boot cihaz listesi oluşturulur:** RBL, **SYSBOOT[4:0]** değerine bakarak hangi cihazları hangi sırayla deneyeceğini belirler (bkz. Bölüm 4).
2. **Process device:** Listedeki sıradaki cihaz denenir. Cihazlar iki türe ayrılır: - **Memory type (bellek türü):** eMMC, SD kart, NAND, SPI flash gibi. MLO bu belleğin içinde **hazır durur**, RBL sadece okur. - **Peripheral type (çevre birimi türü):** UART, USB, Ethernet gibi. MLO kartta yoktur, **dışarıdan** (bir bilgisayardan) gönderilmesi beklenir. Bu yüzden burada **timeout** (zaman aşımı) vardır: Belirli süre içinde imaj gelmezse başarısız sayılır.
3. **Success:** MLO bulunduysa **"Jump to Initial SW"**, yani MLO'ya atlanır ve kontrol ona geçer.
4. **Fail / Timeout:** Listede başka cihaz varsa **bir sonrakine** geçilir.
5. **Dead loop:** Listedeki hiçbir cihazda MLO bulunamazsa RBL sonsuz bir döngüye girer ve kart hiçbir şey yapmaz.

> İpucu
>
> BBB'nin varsayılan boot listesinde **UART0** da vardır. eMMC'de ve SD kartta geçerli bir MLO yoksa RBL sırayı UART0'a getirir ve seri konsoldan X-Modem ile imaj bekler. Bu sırada seri konsolda sürekli `CCCCCCCC...` yazdığını görürsün. Bu çıktı, **"RBL MLO bulamadı"** anlamına gelir.

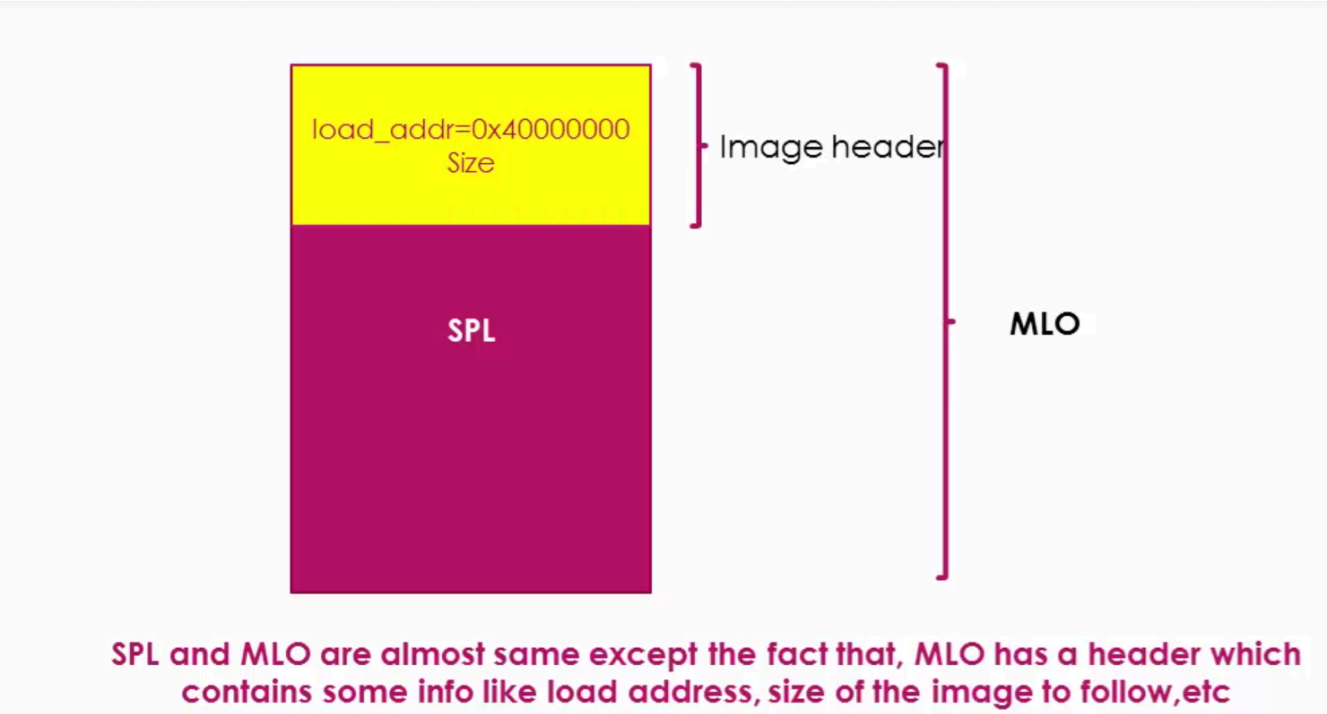
---

2. SPL / MLO (Secondary Program Loader)

- **SPL** = Secondary Program Loader (İkinci Aşama Önyükleyici). TI işlemcilerinde bu dosyanın adı **MLO**'dur (MLO = MMC Loader). SPL ile MLO **neredeyse** aynı şeydir (farkı aşağıda).

- SPL ayrı bir program değildir. **U-Boot'un kaynak kodundan derlenir**, ama sadece gerekli kısımları içeren çok küçük bir versiyonudur. U-Boot'u derlediğimizde hem `MLO` hem de `u-boot.img` dosyaları birlikte oluşur.
- **Nerede çalışır:** SoC içindeki **dahili SRAM** üzerinde (Runs out of internal SRAM). SRAM çok küçük olduğu için SPL de küçük olmak zorundadır.

SPL ile MLO arasındaki fark



- **MLO = Image header + SPL**. Yani MLO, başına küçük bir **başlık (header)** eklenmiş SPL'dir.
- Header'da RBL'in ihtiyaç duyduğu bilgiler bulunur:
- **load_addr:** SPL'in belleğin **hangi adresine** kopyalanacağı
- **Size:** Header'dan sonra gelen SPL kodunun **boyutu**
- RBL önce header'ı okur. Böylece SPL'i nereye kopyalayacağını ve ne kadar veri okuyacağını öğrenir, kopyalamayı yapar ve o adrese atlar.
- U-Boot derlendiğinde önce header'sız SPL (`u-boot-spl.bin`) oluşur. Başına header eklenince `MLO` dosyası elde edilir.

> **Not**
> Slayttaki `0x40000000` adresi bir örnektir. AM335x'te MLO gerçekte dahili SRAM'deki `0x402F0400` adresine yüklenir.

Görevleri:

1. **DDR RAM'i başlatır.** En kritik görevi budur. Saat (PLL) ve DDR zamanlama ayarlarını yapar, böylece 512 MB'lık RAM kullanılabilir hale gelir.
2. Seri konsol (UART0) gibi gerekli birkaç çevre birimini hazırlar. Seri konsolda gördüğün ilk `U-Boot SPL ...` satırı buradan gelir.

3. Üçüncü aşama bootloader'ı, yani **U-Boot**'u (`u-boot.img`) eMMC/SD karttan okuyup **DDR**'a yükler.
4. Kontrolü U-Boot'a devreder.

Soru: AM335x'in RBL'i neden U-Boot'u doğrudan DDR'a yükleyemez?

Cevap: Çünkü RBL, karta **hangi DDR belleğin takılı olduğunu bilmez**. Bilmediği bir belleği de başlatamaz.

- DDR RAM'den veri okuyup yazabilmek için önce SoC içindeki **DDR kontrolcüsünün** ve RAM'in **doğru parametrelerle** ayarlanması gerekir: bellek tipi (DDR2, DDR3, DDR3L...), boyut, hız, zamanlama (timing) değerleri vb.
- Bu parametreler **tamamen karta (ürüne) özeldir**. Aynı AM335x işlemcisini kullanan farklı firmalar farklı bellekler seçebilir:
- X firması: Micron marka **DDR3**
- Y firması: Başka bir markanın **DDR2**'si
- Z firması: Hiç DDR kullanmayabilir
- RBL çip üretilirken ROM'a yazılır. O an çipin hangi kartta, hangi bellekle kullanılacağı belli değildir. Bu yüzden RBL **DDR'ı hiç başlatmaz**. Sadece her işlemcide garanti olarak bulunan **dahili SRAM**'i kullanır.
- U-Boot ise dahili SRAM'e **sığmayacak kadar büyüktür**. Yani RBL'in U-Boot'u yükleyebileceği bir yer yoktur.

Çözüm SPL'dir: SPL, dahili SRAM'e sığacak kadar küçüktür ve kartın DDR ayarlarını **bilir**, çünkü o kart için derlenmiştir. RBL, SPL'i SRAM'e yükler. SPL de DDR'ı başlatıp U-Boot'u DDR'a yükler.

> **Not**
>
> Kendi kartını tasarlayıp BBB'den farklı bir DDR kullanırsan, SPL kaynak kodundaki **DDR parametrelerini** kendi belleğine göre değiştirip SPL'i yeniden derlemen gerekir.

> **Not**
>
> **Saat (clock) ayarları:** RBL'in kodu ROM'da olduğu için saat ayarları **değiştirilemez**. SYSBOOT[15:14] ile sadece karttaki kristalin frekansı bildirilir. İşlemci (MPU), DDR ve çevre birimi frekanslarını asıl kuran **SPL**'dir. SPL'i kendimiz derlediğimiz için bu ayarları kaynak kodda değiştirip yeniden derleyebiliriz. Açılıştan sonra Linux da **cpufreq** ile işlemci frekansını çalışma sırasında değiştirebilir.

---

3. U-Boot (Universal Bootloader)

- Açık kaynak kodlu ve gömülü sistemlerde en yaygın kullanılan bootloader'dır. Üçüncü (ve son) aşama önyükleyicidir.
- **Nerede çalışır: DDR RAM** üzerinde (Runs out of DDR). Artık bellek kısıtı olmadığı için çok daha fazla özellik içerebilir.

Görevleri:

1. Linux Kernel'i çalıştırmak için gereken donanımları (eMMC, SD kart, Ethernet, USB vb.) başlatır.
2. **Linux Kernel** imajını (`zImage` / `uImage`) bir kaynaktan (SD kart, eMMC, ağ üzerinden TFTP vb.) okuyup **DDR**'a yükler.
3. Kartın donanımını tarif eden **Device Tree** dosyasını (Beaglebone Black için `am335x-boneblack.dtb`) DDR'a yükler.
4. Kernel'e **bootargs** (komut satırı parametreleri) iletir. Örneğin root filesystem'in nerede olduğu (`root=`) veya hangi konsolun kullanılacağı (`console=ttyS0,115200`) bu parametrelerle bildirilir.
5. Kontrolü Linux Kernel'e devreder. Bu noktadan sonra U-Boot'un işi biter.

Neden önemli:

- U-Boot'un kendi **komut satırı** vardır. Açılışta seri konsolda geri sayım (`Hit any key to stop autoboot`) sırasında bir tuşa basarsan U-Boot komut satırına (`=>`) düşersin. Buradan bellek okuma/yazma, dosya yükleme, boot ayarlarını değiştirme gibi işlemler yapılabilir.
- Boot ayarları **environment değişkenlerinde** (`bootcmd` , `bootargs` vb.) tutulur ve `printenv` / `setenv` komutlarıyla görüntülenip değiştirilebilir.

Özet Tablo

Aşama	Kim yazar?	Nerede saklanır?	Nerede çalışır?	Ana görevi
RBL	TI (çip üreticisi)	SoC içindeki ROM	ROM	Boot cihazını seçip MLO'yu SRAM'e yüklemek
SPL/MLO	U-Boot projesi (biz derleriz)	eMMC / SD kart	Dahili SRAM	DDR'ı başlatıp U-Boot'u DDR'a yüklemek
U-Boot	U-Boot projesi (biz derleriz)	eMMC / SD kart	DDR	Kernel + DTB'yi yükleyip Kernel'i başlatmak

---

4. Boot Kaynakları ve Boot Sırası (SYSBOOT Pinleri)

AM335x işlemcisi tek bir yerden boot etmek zorunda değildir. **Birçok farklı çevre biriminden** (eMMC, SD kart, UART, USB, SPI flash, NAND, Ethernet...) boot edebilir. Peki RBL hangisinden boot edeceğine nasıl karar verir? Bunu işlemcinin **SYSBOOT** pinleri belirler.

Table 26-7. SYSBOOT Configuration Pins[4]

SYSBOOT[15:14]	SYSBOOT[13:12]	SYSBOOT[11:10]	SYSBOOT[9]	SYSBOOT[8]	SYSBOOT[7:6]	SYSBOOT[5]	SYSBOOT[4:0]	Boot Sequence			
For all boot modes: Crystal Frequency	**For all boot modes**: Set to 00b for normal operation	**For XIP boot**: Muxed or non-muxed device **For NAND boot**: must be 00b	**For NAND and NANDI2C boot**: NAND ECC **For Fast External Boot**: must be 0b	**For XIP boot**: Bus width	**For EMAC boot**: PHY mode	**For all boot modes**: CLKOUT1 output enabled/disabled on XDMA_EVENT_INTR0					
CONTROL_STATUS[23:22]	CONTROL_STATUS[21:20]	CONTROL_STATUS[19:18]	CONTROL_STATUS[17]	CONTROL_STATUS[16]	CONTROL_STATUS[7:6]	CONTROL_STATUS[5]	CONTROL_STATUS[4:0]				
								1st	2nd	3rd	4th
							00000b	Reserved			
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	**For XIP boot**: 00b = non-muxed device 10b = muxed device x1b = reserved	Don't care for ROM code[3]	0 = 8-bit device 1 = 16-bit device	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	00001b	UART0	XIP w/ WAIT[1] (MUX2)[2]	MMC0	SPI0
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	**For NAND boot**: must be 00b	0 = ECC done by ROM 1 = ECC handled by NAND	Don't care for ROM code	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	00010b	UART0	SPI0	NAND	NANDI2C
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	**For XIP boot**: 00b = non-muxed device 10b = muxed device x1b = reserved	Don't care for ROM code	0 = 8-bit device 1 = 16-bit device	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	00011b	UART0	SPI0	XIP (MUX2)[2]	MMC0
00b = 19.2MHz 01b = 24MHz 10b = 25MHz	00b (all other values reserved)	**For XIP boot**: 00b = non-muxed device	0 = ECC done by ROM 1 = ECC	0 = 8-bit device 1 = 16-bit device	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1	00100b	UART0	XIP w/ WAIT[1] (MUX1)	MMC0	NAND

Bu tablo, AM335x'in **Technical Reference Manual (TRM)** dokümanındaki **Table 26-7: SYSBOOT Configuration Pins** tablosudur.

SYSBOOT pinleri nedir?

- **SYSBOOT[15:0]**, işlemcinin 16 adet pinidir. Karta **güç verildiği anda** RBL bu pinlerin seviyesini (1 veya 0) okur.
- Okunan değer, **CONTROL_STATUS** adlı bir register'a kaydedilir. Tablonun 3. satırındaki `CONTROL_STATUS[...]` ifadeleri, her SYSBOOT bitinin bu register'da hangi bite karşılık geldiğini gösterir.
- Bu pinlerin seviyesi kart üzerindeki **pull-up / pull-down dirençleriyle** sabitlenir. Yani boot ayarı, kartı tasarlayan kişi tarafından donanımsal olarak belirlenir.

Tablonun sütunları

Sütun	Ne işe yarar?
SYSBOOT[15:14]	Karttaki kristalin (osilatörün) frekansını bildirir: 19.2 / 24 / 25 / 26 MHz. Beaglebone Black **24 MHz** kristal kullanır (**01b**).
SYSBOOT[13:12]	Normal çalışma için **00b** olmalıdır.
SYSBOOT[11:10], [9], [8]	Sadece belirli boot türlerinde (XIP, NAND) kullanılan ek ayarlar. Diğer boot türlerinde önemsizdir ("Don't care").
SYSBOOT[7:6]	Ethernet (EMAC) üzerinden boot için PHY modu.
SYSBOOT[5]	CLKOUT1 saat çıkışının açık/kapalı olması.
SYSBOOT[4:0]	**En önemli kısım.** Boot modunu, yani **hangi cihazların hangi sırayla** deneneceğini belirler.
Boot Sequence	SYSBOOT[4:0] değerine göre RBL'in deneyeceği cihazlar: **1st** → **2nd** → **3rd** → **4th**.

Boot Sequence (Boot sırası) nasıl çalışır?

Resimde kırmızıyla işaretlenen satırı örnek alalım. **SYSBOOT[4:0] = 00001b** ise boot sırası şöyledir:

```
1st: UART0  →  2nd: XIP (NOR flash)  →  3rd: MMC0 (SD kart)  →  4th: SPI0
```

1. RBL önce **1. cihazı** (UART0) dener.
2. O cihazda geçerli bir boot imajı (MLO) bulamazsa veya cihaz yanıt vermezse **bir sonraki cihaza** geçer.
3. Geçerli bir imaj bulduğu **ilk cihazdan** boot eder ve diğerlerine bakmaz.
4. Dört cihazın hiçbirinde imaj bulamazsa boot başarısız olur.

SYSBOOT[4:0] 5 bit olduğu için 32 farklı kombinasyon vardır. Her biri farklı bir boot sırası tanımlar. Resimde bunların sadece ilk birkaçı görünüyor, tablonun tamamı TRM'de.

Tablodaki boot kaynakları

İsim	Anlamı
MMC0	Beaglebone'da **microSD kart** yuvası
MMC1	Beaglebone'da kart üstündeki **eMMC**
UART0	Seri port üzerinden boot. İmaj bilgisayardan seri hat ile (X-Modem protokolü) gönderilir.
USB0	USB üzerinden boot. Kart, bilgisayara bir ağ cihazı gibi görünür ve imajı oradan alır.
SPI0	SPI ile bağlı harici flash bellekten boot
XIP	e**X**ecute **I**n **P**lace: Kod, NOR flash'a kopyalanmadan doğrudan orada çalıştırılır.
NAND / NANDI2C	Harici NAND flash bellekten boot
EMAC	Ethernet üzerinden ağdan boot

> **Not**
> UART0 ve USB0 boot modları, kartta hiç yazılım olmadığında (örneğin eMMC boşsa veya bozulduysa) kartı **kurtarmak** için kullanılır.

Beaglebone Black'te durum

Beaglebone Black'in boot ayarları, resmi dokümandaki şu tabloda görünür:

SYSBOOT[15:14]	SYSBOOT[13:12]	SYSBOOT[11:10]	SYSBOOT[9]	SYSBOOT[8]	SYSBOOT[7:6]	SYSBOOT[5]	SYSBOOT[4:0]	Boot Sequence			
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	11100b	MMC1	MMC0	UART0	USB0[5]
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	11000b	SPI0	MMC0	USB0[5]	UART0

Durum	SYSBOOT[4:0]	Boot Sequence
S2 basılı değil (varsayılan)	**11100b**	**MMC1 (eMMC)** → MMC0 (SD) → UART0 → USB0
S2 basılı (güç verilirken)	**11000b**	SPI0 → **MMC0 (SD)** → USB0 → UART0

- İki değer arasında sadece **SYSBOOT[2]** biti farklıdır: 11**1**00 → 11**0**00. Boot (**S2**) butonu tam olarak bu biti **0**'a çeker.
- Varsayılan durumda eMMC ilk sırada olduğu için kart **eMMC'deki** hazır Debian'dan açılır.

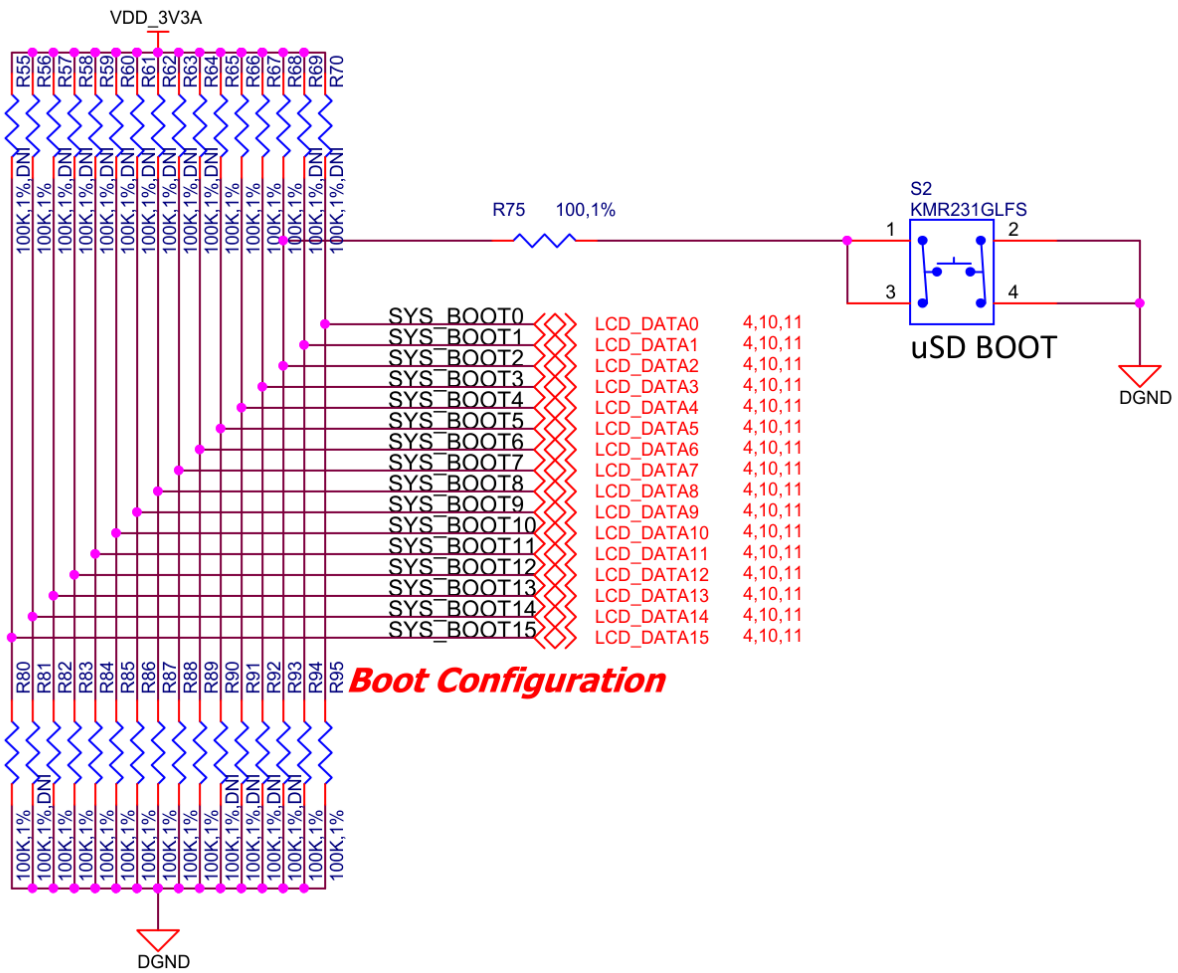
- S2'ye basılıyken eMMC listede **hiç yoktur**. SPI0'da flash olmadığı için o adım atlanır ve kart **SD karttan** açılır. Kendi derlediğimiz MLO/U-Boot/Kernel'i bu yüzden SD karta koyup S2 ile boot edeceğiz.
- Beaglebone'da SYSBOOT pinleri **LCD veri pinleriyle** (LCD_DATA0–15) ortaktır ve P8 header'ına çıkar. Bu pinlere boot sırasında dışarıdan sinyal verilirse boot sırası bozulabilir. Bu yüzden kart açılmadan P8'e bir şey bağlarken dikkatli olmak gerekir.

> **Önemli**
>
> SYSBOOT pinleri **sadece güç verildiği anda** okunur. Kart açıldıktan sonra S2'ye basmak veya Reset butonuna basmak boot sırasını değiştirmez. S2 ile boot etmek için gücü tamamen kesip, S2'yi basılı tutarak tekrar güç vermelisin.

---

5. Schematic'te Boot Devresi

Yukarıda anlatılan boot ayarının kart üzerindeki karşılığı, BBB schematic'inin **6. sayfasındaki** devredir.



> Kaynak: BeagleBone Black Rev C schematic, sayfa 6 (niekiran/EmbeddedLinuxBBB reposundaki `BBB_SCH.pdf`)

Schematic'te dikkat edilecekler:

- Her **SYS_BOOTx** hattına üstte bir pull-up (R55–R70), altta bir pull-down (R80–R95) direnci için yer var. Bunlardan sadece biri karta takılıdır. **DNI** (Do Not Install) yazan direnç kartta **yoktur**.
- Takılı dirençlere bakınca SYS_BOOT[4:0] = **11100b** çıkar. Bu değer, yukarıdaki tablodaki varsayılan değerle aynıdır.
- **S2** butonu (üzerinde **"uSD BOOT"** yazar), **R75** üzerinden **SYS_BOOT2** hattını **GND**'ye bağlar.